

MBAX 6418 Assignment 2

Build a Course Assistant

Work with your teams. Start a new coding-agent task based on the starter prompt below. You are free to modify the prompt to suit your approach. Your team chooses the design, reviews the work, and checks the results.

Using Hermes: After pasting or attaching a prompt, you may need to say: "Build the app described in this prompt." Also share the requirements on the following pages with your agent as you work.

Starter Prompt

Goal

Build a course assistant that answers questions and generates practice quizzes using course materials.

Materials and features

- Use slides, the syllabus, and other course files downloaded from Canvas. Support PDF and other formats, explain what the app accepts, and preserve text and original page/slide images.
- Answer questions using the materials, including diagrams and charts. Acknowledge missing information; never invent answers or citations.
- Let students choose course material, with an optional topic filter, and generate quizzes. Include multiple-choice questions, scores, and explanations. Keep the answer key fixed; hide solutions until students answer or request them.
- For answers and quiz feedback, show the document and page/slide or section, with an actual source excerpt or screenshot. Show original images for visual evidence.

Build

- Use Python for the app and interface. Build hybrid RAG with keyword search, text embeddings, and visual embeddings. Chunk text while preserving source details. Keep text and visual indexes separate; combine search results and rerank candidates. Supply the best evidence, including relevant images, alongside the question and answering instructions.
- Return structured data with separate ``answer`` and ``sources`` fields. Validate the format and check that sources support the answer.
- Possible packages: bm25s (keyword search); langchain-text-splitters (chunking); chromadb (vector storage); gradio (interface).

Class services

- Use the class endpoints at dobolyi.com, ports 9001 and above, for a vision-capable LLM, text/visual embeddings, multimodal reranking, and document parsing. Ask our team for the class endpoint connection details. Follow each service's documented format.
- Store keys in server-side environment variables or a local file. Never expose them in the interface, browser code, logs, errors, screenshots, documentation, test results, or GitHub. Use dummy values in configuration examples.

Work with our team

- Propose a plan and create GitHub issues with completion checks for substantial tasks; let us choose ownership. Help us use branches, commits, pull requests, and reviews. Explain unfamiliar terms.
- Build incrementally, test, save results, and compare one design choice. Include repeatable automated checks. Prepare README.md with setup, screenshots, findings, and limitations. Clearly state what was tested and what remains unchecked.

Build and Test Your App

Use the starter prompt to begin, then work through the requirements below with your agent. Your team can choose its own interface and implementation. Expect to revise the app as you discover missing features or incorrect results.

1. Let users manage course materials

Users must be able to **add and remove documents through the app**. Loading the same file twice should not duplicate it. When a document is removed, remove its searchable content too, so later answers do not rely on it.

Working with PowerPoint: Direct upload of PowerPoint (PPTX) files is preferred. Automated conversion can take extra work; ask your agent about using Python with [LibreOffice](https://libreoffice.com) to convert slides to PDFs that can be rendered as images. Manually exporting a deck to PDF before uploading is an acceptable workaround. Document the supported formats and conversion steps, including any extra software needed, and check that converted slides still look correct.

2. Retrieve, show, and explain visual content

When a user asks a question about the slides, your app must retrieve relevant slide images through RAG and **show pictures of the actual slides** alongside its answer, with the document name and slide number. Users must also be able to ask the app to describe or explain slide content, including pictures, diagrams, and charts, based on the retrieved materials.

Try it: Ask your app to find the meme about **Vibe Coding on "Prod"** in the Week 2 course slides. The app should retrieve and display the correct slide, then summarize what its image and text show. Also try questions about diagrams or charts. Check the displayed source against the original slide and make sure it supports the answer.

3. Check quiz generation and feedback

Verify that generated quiz questions are answerable from the selected material, that solutions stay hidden until the user answers or requests the answer, and that scores match the answer key. Check that explanations are useful and their sources support the correct answers.

4. Test the app thoroughly

Test thoroughly for bugs and missing features. Use **manual testing** and ask your agent to add **unit tests** (checks of individual functions) and **end-to-end tests** (checks of complete workflows). Exercise adding and removing files, answering text and image questions, and generating and scoring quizzes. Include missing information, repeated uploads, and unavailable model services. Fix the problems you find and rerun the relevant tests.

Check **light and dark modes**, if available, including the readability of text, controls, and source images.

Evaluate, Document, and Submit

Evaluate answer quality

Create a set of **5-10 questions** you might ask a course assistant. Cover the syllabus and slide text, and include **at least two questions that require visual information** (including the meme example). Include **one question the materials cannot answer** to check whether the app acknowledges missing information.

Use some of these questions to compare **two approaches to one part of your RAG system**, such as reranking on versus off or two chunking methods. Use the same questions and course files for both. Record whether each answer is correct, whether its sources support it, and how long it takes. In your **README.md report**, include your question set, comparison results, and interpretation, explaining which approach you would keep and why.

Make the app usable by someone else

Your submission must include everything I need to deploy the app and test it independently. Provide the code, dependency information, and clear instructions to install, configure, and launch it. List any additional software, such as LibreOffice, and explain how to supply API keys without committing them. A teammate should follow the documentation from a fresh repository copy; fix any missing steps they discover.

Write your README.md report and documentation

Use **README.md** as the starting point for both your report and the app's documentation.

Include:

- **Setup and use:** How to install and launch the app, configure endpoints, add/remove documents, ask questions, generate quizzes, and rerun tests. Explain supported file formats, any manual conversion, and known limitations.
- **An SVG architecture diagram:** Save the final diagram as an **.svg file in the repository and embed it in README.md** so it displays on GitHub. Show document preparation, keyword/text/image retrieval, reranking, and answer generation. Explain the components, models/services, and where they run. The diagram must match your final app.
- **Results and findings:** Present the questions, comparison results, and interpretation described in **Evaluate answer quality** above, and discuss at least one limitation or failure you investigated.
- **At least two app screenshots:** An answer displaying a supporting slide image and quiz feedback with sources. Include the SVG diagram in addition to these screenshots.

The agent may draft the documentation and diagram. Your team must check that they describe the finished app accurately, verify the claims against your results, and explain your findings in your own words.

Work together and submit

Use **one shared GitHub repository**, issues to organize substantial tasks, and pull requests to review changes before combining them. Everyone must make a meaningful contribution and review a teammate's work.

Submit **one repository link per team on Canvas**. I will use the most recent commit made before the assignment closes. Include code, README.md and other documentation, the SVG diagram, screenshots, saved results, dependencies, and configuration instructions. Private repositories are fine; give me access before submitting. Confirm the link works. **Keep API keys out of the repository.**